ParkChain

A Smart-Contract-Based Urban Parking and EV Charging Network

Emily Lim Gina Lin Vladyslav Nyzhnyk Ian Virchenko

Technische Universität Berlin

Introduction to ParkChain

Urban parking and EV charging services are often fragmented across multiple providers. Users must manage different reservation systems and payment methods, while operators require trustworthy accounting and revenue management.

ParkChain addresses this challenge through a blockchain-based platform where members purchase a monthly membership, receive ParkCredits, and use them across all registered operators.

What makes ParkChain different?

ParkChain extends the common membership-and-credit model with a complete reservation lifecycle. Reservations are time-based, checked for overlaps, and finalized through check-in and check-out. Smart contracts automatically enforce grace periods, no-show fees, and overstay charges.

Stakeholders

Member

Purchases memberships, receives ParkCredits, reserves parking slots, checks in, checks out, and consumes charging services.

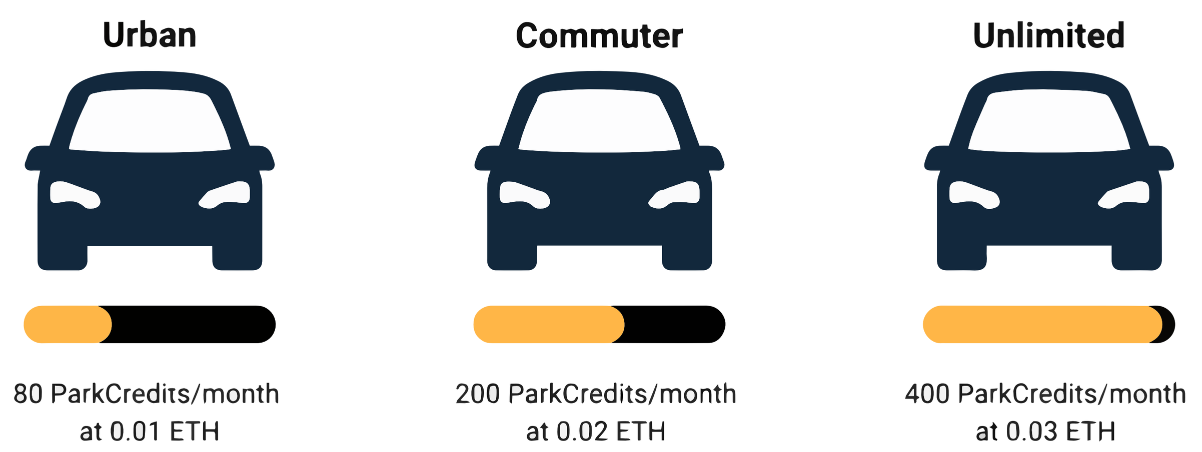
Operator

Provides parking and charging infrastructure, configures pricing models, and receives revenue through the platform treasury.

Administrator

Registers operators, manages membership plans, configures system-wide parameters, and supervises platform governance.

Membership Tiers



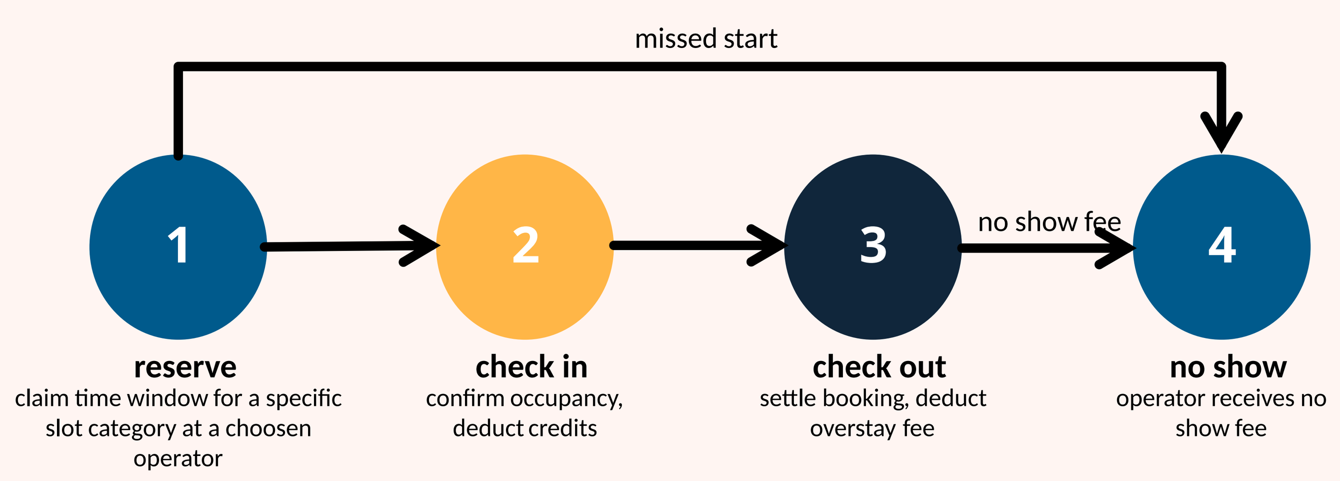
System Concept

ParkChain uses Ethereum smart contracts and ERC-1155-compatible ParkCredits as the platform's payment and accounting mechanism. Members purchase monthly memberships that mint ParkCredits to their wallet. These credits are then used to reserve parking slots or EV charging bays across all registered operators. The platform supports multiple membership tiers with different:

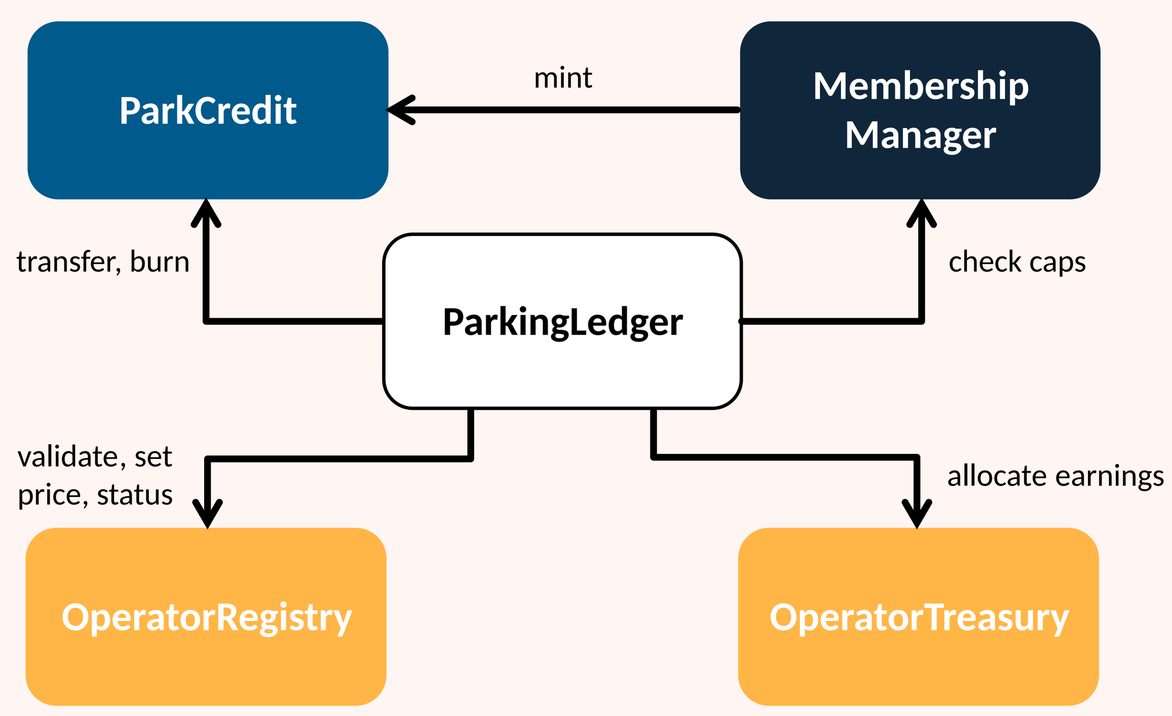
- credit allowances,
- monthly occupancy limits,
- and usage restrictions per operator and slot category

Reservation lifecycle

Parking follows a four-phase lifecycle:



Contract Modules



Development Framework and Tools

TypeScript React Solidity OpenZeppelin Hardhat

Challenges and solutions

Challenges

- **Merge Conflicts** caused by parallel development of smart contracts and frontend components
- **Smart Contract Address Changes** during deployment complicate testing and frontend integration
- **Frontend Complexity** due to blockchain interactions, wallet connections, and transaction handling
- **Volatile ETH Price** affects predictable membership pricing and operator payments
- **Admin as User** creates role-management and authorization challenges
- **Gas Costs** may become expensive on Ethereum mainnet

Solutions

- Regular synchronization meetings
- Clear contract responsibilities
- Incremental integration and testing
- Router contract to store the adresses of other contracts

Progress

- BPMN modeling
- Stakeholder and smart contracts analysis
- Implement tokens, operator registry and treasury
- Implement tests and running on local devices
- Initial frontend structure

- Complete ParkingLedger
- Complete MembershipManager
- Implement reservation logic
- Integration tests between contracts
- Frontend integration